

Claude Code Sourcemap 项目

Agent 与 LLM 架构图导读

0. 这份文档的定位

这份文档不是简单重复 [agent-llm-engineering-analysis.md](#)。

前一份文档更像“工程思想分析”，回答的是：

- 它为什么这样设计
- 每一层在解决什么问题
- 各种做法的收益、代价和边界是什么

而这一份文档更像“带你顺着图和时序真正走一遍系统”。

它重点回答的是：

- 一个请求从哪里进入
- 中途经过哪些关键节点
- Prompt、Tool、Memory、Agent 协作分别在什么时候介入
- 哪些地方是控制点，哪些地方是状态点，哪些地方是副作用点

如果你以前看 Agent 系统的图经常有一种感觉：

图我看懂了，但我还是不知道代码里到底怎么跑起来。

那这份文档就是专门解决这个问题的。

为了便于理解，下面我会反复强调四种“看图的方法”：

1. 看“谁发起”。
2. 看“状态存在哪里”。
3. 看“副作用在哪里发生”。
4. 看“结果怎样回流到下一轮推理”。

只要你抓住这四件事，复杂系统就不会只剩一堆名词。

1. 总体模块图

先把整个系统压成一张图：





1.1 这张图最重要的不是“模块名”，而是“系统中心”

很多人第一次看这类系统，会下意识觉得模型 API 是中心。

但这个项目真正的中心不是 Anthropic API，而是 query loop。

也就是说，系统真正围绕的是这样一个循环：

1. 组织上下文。
2. 让模型判断下一步。
3. 如果要行动，就执行行动。
4. 把行动结果再放回上下文。
5. 继续判断，直到这一轮完成。

这意味着：

- Prompt 不是前置静态配置，而是循环里的输入控制器。
- Tool 不是外挂功能，而是循环里的行动执行器。
- Memory 不是单独数据库，而是循环里的长期补充信息源。
- Compact 不是可选优化，而是循环能否长期稳定运行的必要机制。

所以你看源码时，最值得盯住的不是哪个工具函数最花哨，而是：

谁在驱动这个循环，谁在修改这个循环，谁在为下一个循环准备条件。

2. 从入口看系统：为什么 main.tsx 不只是启动文件

关键文件：

- restored-src/src/main.tsx

2.1 入口层到底在做什么

很多项目的入口文件只是“把应用跑起来”。
但在这个项目里，入口层已经在决定后面很多 Agent 行为的边界。

它处理的并不只是技术启动，还包括：

- 当前用户身份和认证状态
- 功能开关是否启用
- 哪些插件、技能、MCP 服务可用
- Agent 定义如何加载
- 当前是否处于 coordinator 模式
- 初始配置与运行环境怎样注入到后续会话

这一步为什么重要？

因为模型并不是在一个“天然完整的世界”里工作。
模型看到的工具、角色、权限、模式，其实都是运行时先搭好的。

所以入口层的本质不是“启动程序”，而是：

搭建 Agent 即将工作的操作环境。

2.2 这里最值得学的工程思想

很多团队做 Agent 时，会把精力都放在 Prompt 和工具上，忽略运行环境装配。
结果到了后面会出现几个常见问题：

- 开关太多，但没有统一入口管理
- 不同模式下工具边界不一致
- 插件和核心逻辑耦合得很死
- Agent 定义散落在各处，很难验证和覆盖

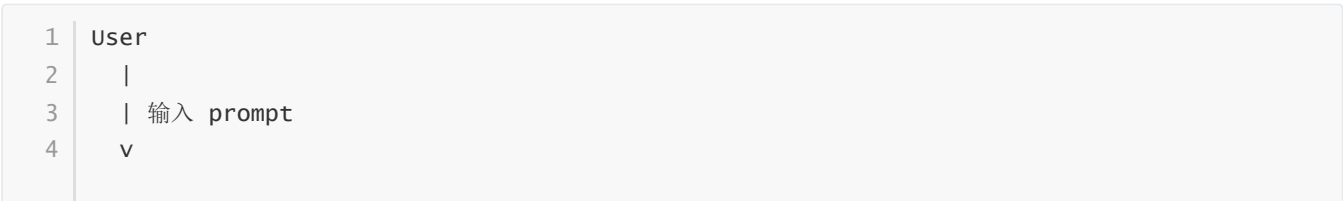
这个项目的做法更成熟的地方在于：

- 它把“可见能力”前移到启动层和装配层处理
- 它让后面的推理循环基于一个已经整理好的运行环境工作
- 它尽量避免在 query loop 深处再到处判断“这个东西能不能用”

换句话说，入口层的价值是减少后续复杂度。

3. 请求时序图：一条普通用户请求是怎么走的

3.1 标准单轮流程



```

5  QueryEngine.submitMessage()
6  |
7  | 1. 重置当前 turn 的临时状态
8  | 2. 准备 permission / app state / cwd
9  v
10 fetchSystemPromptParts()
11 |
12 | 3. 收集 system prompt 各部分
13 v
14 buildEffectiveSystemPrompt()
15 |
16 | 4. 组装真正送给模型的系统上下文
17 v
18 processUserInput()
19 |
20 | 5. 处理 slash command / attachments / mentions / 特殊输入
21 v
22 query()
23 |
24 | 6. 发起模型请求
25 | 7. 流式接收 assistant 输出
26 | 8. 如果出现 tool_use -> 进入工具编排
27 v
28 runTools()
29 |
30 | 9. 并发或串行执行工具
31 | 10. 产出 tool_result messages
32 v
33 query() 再入
34 |
35 | 11. 把 tool_result 回注给模型继续推理
36 v
37 最终 assistant 响应
38 |
39 | 12. stopHooks / compact / memory extraction / persistence
40 v
41 User sees output

```

3.2 这张时序图真正该怎么看

很多人看时序图时，只会顺着箭头读步骤。

但在 Agent 系统里，更重要的是识别四种角色：

- `QueryEngine.submitMessage()`：本轮请求的总控入口
- `fetchSystemPromptParts()` 与 `buildEffectiveSystemPrompt()`：上下文准备器
- `query()`：推理与行动的主循环
- `runTools()`：副作用执行器

你可以把它理解成一条流水线，但更准确地说，它是一条“带回流的流水线”。

为什么说“带回流”？

因为工具执行不是终点。工具执行完以后，结果还要重新进入模型推理。
这和普通后端调用完全不同。

普通后端链路常常是：

- 请求进来
- 执行业务逻辑
- 返回结果

而这里是：

- 请求进来
- 模型决定下一步
- 系统执行这一步
- 执行结果成为模型下一步思考材料

所以你不能把 Tool Calling 当成一个孤立函数。
它实际上是模型认知链条中的一个中间动作。

3.3 源码里真正的控制点在哪里

如果你想顺着代码跟一遍，建议按这个顺序看：

1. `restored-src/src/QueryEngine.ts`
2. `restored-src/src/utils/queryContext.ts`
3. `restored-src/src/utils/systemPrompt.ts`
4. `restored-src/src/utils/processUserInput/processUserInput.ts`
5. `restored-src/src/query.ts`
6. `restored-src/src/services/tools/toolOrchestration.ts`
7. `restored-src/src/query/stopHooks.ts`

这一串文件分别负责：

- 会话入口与状态组织
- system prompt 片段收集
- system prompt 最终组装
- 用户输入预处理
- 模型循环主逻辑
- 工具执行编排
- 本轮结束后的后台收尾

这就是所谓“源码级链路”。

当你理解了这串责任分工，再看任何局部功能，都更容易定位它在全局中的位置。

4. Prompt 组装图：上下文不是一段文字，而是拼装出来的

4.1 Prompt 组装分层

```
1  getSystemPrompt() / fetchSystemPromptParts()
2      |
3      v
4  +-----+-----+-----+
5  | Static Parts | Dynamic Sections | Attachments |
6  | Intro / rules | memory / env / MCP | reminders / deltas |
7  | tool policies | language / mode | skill context |
8  +-----+-----+-----+
9      |
10     v
11  buildEffectiveSystemPrompt()
12      |
13     v
14  Final model context
```

4.2 为什么这里不能理解成“写一段大 Prompt”

如果你把 Prompt 理解成一大段 system prompt，你就会误解这个项目最关键的工程手法。

这里真正发生的是三件事：

1. 把稳定规则和波动信息拆开。
2. 把高频变化的部分尽量移出稳定缓存前缀。
3. 让不同来源的信息以不同形式进入模型。

这背后的目标不是“Prompt 更长更强”，而是：

- 更稳定
- 更可维护
- 更利于缓存
- 更适合长会话演进

4.3 源码里是怎么一步步实现的

最值得跟的是这条链路：

- `QueryEngine.ts` 里调用 `fetchSystemPromptParts()`
- `queryContext.ts` 收集可用的系统上下文片段
- `systemPrompt.ts` 中的 `buildEffectiveSystemPrompt()` 做最终拼装
- `attachments.ts` 把部分动态信息以 attachment 而不是固定 system prompt 的方式送进上下文

为什么要这样拆？

因为系统里有两类完全不同的信息：

- 一类是长期稳定规则，比如工具使用原则、行为边界、格式要求

- 一类是短期波动信息，比如当前模式、临时提醒、记忆注入、上下文差异

如果全部硬塞到一段 system prompt 里，就会出现几个问题：

- 每次变化都破坏缓存
- 规则和临时信息混在一起，不容易调试
- 很难判断某条行为是由哪个 Prompt 片段触发的

所以这里真正学到的是：

Prompt 设计不是文学创作，而是上下文供给系统设计。

5. Tool Calling 图：工具调用不是一次函数跳转，而是一个小运行时

5.1 Tool 调用时序

```

1  assistant message
2  |
3  | tool_use blocks
4  v
5  runTools()
6  |
7  | partitionToolCalls()
8  | 判断每个调用是否并发安全
9  v
10 +-----+-----+
11 | 并发安全 batch          | 非并发安全 batch          |
12 | runToolsConcurrently() | runToolsserially()          |
13 +-----+-----+
14 |
15 | v
16 | runToolUse()
17 | |
18 | | 权限检查 canUseTool
19 | | 具体工具执行
20 | | 结果包装成 message
21 | v
22 | tool_result messages
23 | |
24 | v
25 | 回到 query loop
  
```

5.2 为什么这里一定要有“编排层”

很多 demo 系统做工具调用，往往就是：

- 模型返回一个函数名和参数
- 代码直接执行
- 把结果字符串拼回去

这在 demo 阶段很好用，但到真实系统里很快就不够了。

因为真实工具调用要面对的不是“能不能调用”，而是：

- 会不会互相影响
- 是否有副作用
- 是否需要权限确认
- 是否允许并发
- 失败时如何回写
- 结果如何和会话协议对齐

所以 `toolorchestration.ts` 的价值，不在于“帮忙调函数”，而在于：

它把模型的动作意图，变成一个受控、可治理、可回放的执行流程。

5.3 这条链路在代码里是怎样落地的

建议结合下面几个位置一起看：

- `restored-src/src/query.ts`
- `restored-src/src/services/tools/toolorchestration.ts`
- `restored-src/src/Tool.ts`
- `restored-src/src/services/mcp/client.ts`

它们大致的责任是：

- `query.ts`：发现模型发出了 `tool_use`
- `toolorchestration.ts`：决定工具应该怎样被编排执行
- `Tool.ts`：定义工具能力边界、元数据、协议和实现入口
- `mcp/client.ts`：把外部 MCP 能力接到统一工具系统里

这里最值得你记住的一点是：

工具执行结果最终不是“某个函数返回值”，而是“会话消息的一部分”。

这件事看起来小，实际上非常关键。

因为只有这样，compact、resume、多 Agent 回流、日志记录、状态恢复才能共用同一套消息协议。

6. Compact 时序图：这个系统怎么防止长会话崩掉

6.1 Compact 触发与回写

```
1 query loop
2   |
3   | 检查 token 使用与 compact 条件
4   v
5 compactConversation() / sessionMemoryCompact()
6   |
7   | 1. 选择要压缩的历史消息
8   | 2. 生成 summary / compact boundary
```



```
9 | 3. 形成压缩后的消息集合
10 | 4. 清理需重建的上下文状态
11 v
12 后续 query 继续运行
```

6.2 为什么 compact 不是“做个摘要”这么简单

长会话 Agent 最常见的误解之一，就是把 compact 当成一个纯文本总结功能。

其实在这个项目里，compact 更像一次“会话状态重构”。

它会同时影响：

- 历史消息结构
- 后续 Prompt 的形成方式
- 哪些动态信息必须重新注入
- 哪些缓存应该保留，哪些状态应该清空

这就是为什么相关逻辑不只出现在一个摘要函数里，而是涉及：

- `services/compact/compact.ts`
- `services/compact/sessionMemoryCompact.ts`
- `query/stopHooks.ts`

6.3 为什么 compact 后必须 reinjection

这是长会话系统特别容易出 bug 的地方。

因为压缩历史意味着你主动丢弃了一部分原始上下文。

如果你压缩后不重新注入关键骨架，模型会出现这些问题：

- 忘掉工具边界
- 忘掉模式状态
- 忘掉部分记忆提示
- 忘掉之前的关键任务约束

所以 compact 之后真正正确的做法不是：

- “把旧消息删掉就好了”

而是：

- “删掉旧负担，再把关键骨架重新补回去”

这就是这个项目在会话治理上比较成熟的地方。

7. Memory 与知识流图：它是如何把会话变成长期资产的

7.1 从会话到长期记忆

```
1  当前会话 messages
2  |
3  | turn 结束 / stopHooks
4  v
5  extractMemories()
6  |
7  | forked extraction agent
8  v
9  memory files (*.md)
10 |
11 | 更新索引与分类
12 v
13 未来会话通过 memdir / relevant recall 再次读回
```

7.2 从多 session 到 consolidation

```
1  多个 session transcripts / memory files
2  |
3  | 达到时间门槛 + 会话数量门槛
4  v
5  autoDream()
6  |
7  | consolidation agent
8  v
9  重写 / 合并 / 提炼 memory files
```

7.3 为什么记忆这里分成两种不同时间尺度

这个项目里，`extractMemories` 和 `autoDream` 不只是两个功能名，而是两种不同时间尺度的治理机制。

- `extractMemories`：面向“刚发生过什么，哪些值得沉淀”
- `autoDream`：面向“过一段时间后，哪些记忆需要合并、清理、升格”

这说明它不是把 memory 当成简单日志，而是在做知识沉淀流程。

如果只做第一步，不做 consolidation，会出现：

- 记忆文件越来越碎
- 重复事实越来越多
- 老结论和新结论互相冲突

如果只做 consolidation，不做细粒度 extraction，又会出现：

- 很多细节从未被写入
- 真正重要的长期偏好没有原始材料可整理

7.4 源码里怎么对应这些流程

建议把下面几组文件连起来看：

- `restored-src/src/query/stopHooks.ts`
- `restored-src/src/services/extractMemories/extractMemories.ts`
- `restored-src/src/services/autoDream/autoDream.ts`
- `restored-src/src/memdir/*`
- `restored-src/src/utis/claudemd.ts`

这一组共同组成了：

- 记忆提取触发
- 记忆落盘
- 记忆检索
- 记忆整理
- 规则型知识注入

所以这里的 Memory，不只是“存储层”，更像“长期上下文运营层”。

8. 多 Agent 协作图：真正复杂的不是 spawn，而是回流

8.1 Coordinator 与 Worker 的协作流

```
1 主 Agent / Coordinator
2  |
3  | 拆任务、选角色、设边界
4  v
5  runAgent() / startInProcessTeammate()
6  |
7  | worker / teammate 执行
8  v
9  mailbox / task state / notifications
10 |
11 | 结果回流、状态回流、中断与恢复
12 v
13 Coordinator 汇总并对用户负责
```

8.2 为什么复杂点不在 spawn，而在状态回流

很多人一提多 Agent，就会把注意力放在“如何起多个模型”。但那其实是最简单的一步。

真正难的是：

- 每个 Agent 拿到什么上下文
- 谁负责最终结论
- Worker 中途状态如何被外部看到

- 结果通过什么协议回流
- 中断、恢复、重复执行如何避免混乱

这个项目之所以更像工业化系统，是因为它没有把多 Agent 做成“单次调用多个模型”。它做的是带任务状态、消息回流和生命周期管理的协作系统。

8.3 代码里主要看哪几个点

建议优先看：

- `restored-src/src/tools/AgentTool/runAgent.ts`
- `restored-src/src/tools/AgentTool/loadAgentsDir.ts`
- `restored-src/src/tools/AgentTool/prompt.ts`
- `restored-src/src/coordinator/coordinatorMode.ts`
- `restored-src/src/utils/swarm/inProcessRunner.ts`
- `restored-src/src/utils/forkedAgent.ts`

这些文件分别大致对应：

- Agent 实际执行
- Agent 定义装载
- Agent Prompt 约束
- 协调模式与角色边界
- in-process teammate 持续运行循环
- 分叉上下文与缓存安全继承

这里最值得学的地方是：

多 Agent 系统不是“多开几个会说话的模型”，而是“建立可回流、可观测、可协调的任务组织结构”。

8.4 为什么 worker 结果用 notification / task 消息回流

因为真实多 Agent 系统里，worker 很少是一次性同步返回。

它可能会出现这些情况：

- 先发进度，再发结果
- 结果不完整，需要补充追问
- 任务被打断，需要恢复
- 多个 teammate 同时工作，需要统一汇总

如果你只设计一个“函数返回值”，这些场景都会很难扩展。

而消息化回流的好处是：

- 统一协议
- 统一日志
- 统一中断恢复
- 统一状态呈现

这也是这个项目从 demo 走向系统工程的关键一步。

9. 四种关键场景的时序解读

这一节不是再讲新机制，而是把前面那些机制真正放进场景里。

9.1 场景一：普通代码修复

典型链路是：

1. 用户提出修改需求。
2. `QueryEngine.submitMessage()` 建立本轮状态。
3. `processUserInput()` 解析输入。
4. `buildEffectiveSystemPrompt()` 组装本轮上下文。
5. `query()` 发起模型请求。
6. 模型发出读文件、搜索、编辑等工具调用。
7. `runTools()` 负责编排执行。
8. tool result 回注给模型。
9. 模型给出最终改动与解释。
10. `stopHooks` 处理后续收尾。

这里最值得学的，是“推理”和“执行”如何交替推进，而不是一次性完成。

9.2 场景二：上下文越来越长

典型链路是：

1. 会话不断积累消息和 tool result。
2. 系统监测到 token 或结构压力。
3. `compactConversation()` 或 `sessionMemoryCompact()` 触发。
4. 历史被压缩，关键骨架重新注入。
5. 新的 query loop 在较轻上下文上继续。

这说明长会话能力不是模型天然给的，而是编排系统托起来的。

9.3 场景三：用户偏好被沉淀到未来

典型链路是：

1. 本轮对话结束。
2. `stopHooks.ts` 触发后台任务。
3. `extractMemories()` 从会话中提取长期价值信息。
4. 记忆落盘到 memdir。
5. 未来会话开始时，相关记忆再被召回注入。

这说明“记住用户”不是模型自己会记，而是系统主动做沉淀和检索。

9.4 场景四：复杂任务拆给多个 Agent

典型链路是：

1. 主 Agent 判断任务过大或适合分工。
2. `runAgent()` / `startInProcessTeammate()` 启动子执行体。
3. worker 基于自己的上下文和边界工作。
4. 中间过程通过 mailbox / task state / notification 回流。
5. coordinator 汇总后对用户给最终答复。

这说明真正高阶的不是“能拆”，而是“拆完还能收得回来”。

10. 如果你自己画架构图，最应该画哪几张

如果你想把一个 Agent 系统讲清楚，不需要一开始就画特别大的全景图。
更有效的做法，是优先画下面五张。

10.1 分层图

目的：回答“系统分成哪些层，每层职责是什么”。

适合讲：

- 入口与环境装配
- 会话状态层
- query loop
- prompt 系统
- tool runtime
- memory / knowledge
- multi-agent orchestration

10.2 单轮时序图

目的：回答“一条请求到底怎样走完一圈”。

这是最关键的一张图，因为它决定你是不是在真正理解 Agent Runtime。

10.3 Tool Calling 图

目的：回答“模型发出动作请求后，系统如何安全落地执行”。

这张图最能体现工程成熟度，因为它直接暴露权限、并发、副作用治理能力。

10.4 Memory 流图

目的：回答“临时会话如何变成长期资产，又怎样被取回来”。

如果没有这张图，很多团队会把 memory 误解成“存聊天记录”。

10.5 多 Agent 协作图

目的：回答“任务如何拆分、状态如何回流、责任如何闭环”。

如果没有这张图，多 Agent 很容易被画成几个框加几根箭头，结果完全讲不出真正复杂的地方。

11. 你看这套架构图时，真正应该学到什么

看到这里，最重要的收获不应该只是“我记住了这些文件名”。

你真正应该学到的是下面几件事。

11.1 先找循环，再找模块

复杂 Agent 系统不是模块堆砌出来的，而是围绕一个核心循环长出来的。

在这个项目里，这个核心循环就是 query loop。

11.2 先找状态点，再找算法点

很多人只盯着 Prompt 和模型调用。

但系统稳定性更多来自状态治理。

你要优先看清：

- 会话状态存在哪里
- 工具副作用在哪里发生
- compact 之后哪些状态被重建
- memory 怎样落盘与召回

11.3 先找回流路径，再找局部功能

Agent 系统和普通工具系统最大的差别，就是几乎所有局部动作都要回流到后续推理。

所以你每看一个功能，都应该问：

它的结果怎样进入下一轮决策？

11.4 工程价值往往藏在“中间层”

很多炫目的功能看起来都在模型层。

但真正决定系统是否可扩展的，往往是中间层：

- Prompt 组装层
- Tool 编排层
- compact 层
- memory 提取与整理层

- multi-agent 协调层

这些层不一定最显眼，但它们最像“工业化底盘”。

12. 关键源码索引

入口与装配：

- `restored-src/src/main.tsx`

会话与请求入口：

- `restored-src/src/QueryEngine.ts`
- `restored-src/src/utils/queryContext.ts`
- `restored-src/src/utils/processUserInput/processUserInput.ts`

主执行循环：

- `restored-src/src/query.ts`
- `restored-src/src/query/stopHooks.ts`

Prompt 系统：

- `restored-src/src/constants/prompts.ts`
- `restored-src/src/constants/systemPromptSections.ts`
- `restored-src/src/utils/systemPrompt.ts`
- `restored-src/src/utils/attachments.ts`

Tool Runtime：

- `restored-src/src/Tool.ts`
- `restored-src/src/services/tools/toolOrchestration.ts`
- `restored-src/src/services/mcp/client.ts`

Compact / 长会话治理：

- `restored-src/src/services/compact/compact.ts`
- `restored-src/src/services/compact/sessionMemoryCompact.ts`

Memory / Knowledge：

- `restored-src/src/memdir/memdir.ts`
- `restored-src/src/memdir/findRelevantMemories.ts`
- `restored-src/src/utils/claudeMd.ts`
- `restored-src/src/services/extractMemories/extractMemories.ts`
- `restored-src/src/services/autoDream/autoDream.ts`

多 Agent：

- `restored-src/src/tools/AgentTool/loadAgentsDir.ts`

- `restored-src/src/tools/AgentTool/runAgent.ts`
- `restored-src/src/tools/AgentTool/prompt.ts`
- `restored-src/src/coordinator/coordinatorMode.ts`
- `restored-src/src/utils/forkedAgent.ts`
- `restored-src/src/utils/swarm/inProcessRunner.ts`